SIMO

Scalable Ingredient Management Organizer

Versione 2.0

Specifiche di Progettazione Software, Modello Matematico e Architettura dei Sottosistemi

Manuale di Riferimento per Ingegneri del Software, Architetti di Sistema e Addetti ai Lavori

Stack Tecnologico Primario

Kotlin 2.0 • Android SDK (API 24–37) • Jetpack Compose (Material 3)
Android Room (SQLite) • Kotlin Coroutines & Reactive StateFlow
AndroidKeyStore Hardware-Backed Crypto (AES-256 GCM)
Dual-Engine Cloud AI (Google Gemini v1beta & OpenAI ChatGPT API)
Multi-Page Vectorial PDF Engine (ISO 216 A4) • Resa Termica & Nutrizione UE

Document ID: SIMO-ENG-SPEC-2026-V2 **Stato:** Release Ufficiale v2.0
Autore: Francesco Panattoni
Data: 29 ottobre 2026

Indice

1 Introduzione **3**
1.1 Problem Statement & Contesto di Dominio . . . 3
1.2 Requisiti Non Funzionali e Vincoli Architetturali . . . 3

2 Architettura del Software e Pattern Architetturali **5**
2.1 Unidirectional Data Flow (UDF) e State Immutability . . . 5
2.2 Concorrenza e Thread Safety . . . 6
2.3 Lifecycle Optimization: Warm-up Asincrono e Pre-caching all'Avvio . . . 6

3 Domain Engine: Teoria dei Grafi e Calcolo Matematico **7**
3.1 Formalizzazione del Ricettario come DAG Pesato . . . 7
3.2 Algoritmo di Rilevamento Cicli Preventivo (`wouldCreateCycle`) . . . 7
3.3 Algoritmo di Espansione ad Albero Scalato (`buildScaledTree`) . . . 8
3.4 Modello di Reverse Scaling . . . 8
3.5 Quantizzazione e Sistema delle Unità di Misura . . . 9
3.6 Modello Analitico del Food Cost . . . 9
3.7 Coerenza Metrica tra Food Cost e Carrello della Spesa . . . 9
3.8 Tipografia e Algoritmo di Line-Breaking Antifrattura . . . 10
3.9 Stabilità Tipografica dei Badge Allergeni e Prevenzione della Compressione Orizzontale 10
3.10 Equivalenza Culinaria Massa-Volume (1 : 1) nel Food Cost e nella Spesa . . . 11
3.11 Modello Matematico della Variazione di Massa in Cottura (Resa Termica) . . . 11
3.12 Modello di Calcolo della Dichiarazione Nutrizionale UE (Regolamento UE n. 1169/2011) . . . 11
3.13 Algoritmo di Concentrazione e Diluizione Nutrizionale Post-Cottura . . . 12
3.14 Regole di Conservazione Gastronomica dell'Unità di Resa . . . 12

4 Strato Dati, Room Database e Persistenza Reattiva **13**
4.1 Schema Relazionale e Normalizzazione . . . 13
4.2 Polimorfismo Relazionale degli Item . . . 13
4.3 Prevenzione dei "Dangling Pointers" Relazionali . . . 13
4.4 Evoluzione dello Schema e Migrazioni Database . . . 14
4.5 Pattern di Modifica Reattiva: `EditableComponent` . . . 15
4.6 Algoritmo di Riordinamento Reattivo Continuo (Drag & Drop) e Spostamento Discreto . . . 15

5 Sicurezza Hardware-Backed e Crittografia KeyStore **17**
5.1 Architettura del Modulo `CryptoUtils` . . . 17
5.2 Parametri Crittografici e Modalità Operativa . . . 17

6 Integrazione Cloud AI: Architettura Dual-Engine (Gemini & ChatGPT) **18**
6.1 Architettura Dual-Provider e Protocollo di Rete . . . 18
6.2 Selezione del Modello (Model Tiering) . . . 18
6.3 Modalità Operative Non Distruttive dell'Assistente AI . . . 19
6.4 Prompt Engineering & Strutturazione JSON Forzata . . . 19
6.5 Sanitizzazione Output & Motore di Fallback Euristico . . . 19
6.6 UI State Handling, Indicatori Lineari e Layout Scrollabile . . . 20

7 Cooking Mode, Concorrenza e Sottosistemi Hardware **20**
7.1 Integrazione Window Manager: Flag "Keep Screen On" . . . 20
7.2 Motore NLP Deterministico: `CookingTimerExtractor` . . . 20
7.3 Macchina a Stati dei Timer Concorrenti e Architettura Persistente . . . 20
7.4 Interfacciamento con Hardware HAL: Audio & Vibrazione . . . 21
7.5 Componente Modale Non Intrusivo: `QuickAddTimerDialog` . . . 21
7.6 Gestione Layout Insets e Risoluzione Occlusioni di Sistema . . . 21

8 Rendering Grafico e Generazione PDF **22**
8.1 Architettura del Modulo `RecipePdfExporter` . . . 22
8.2 Geometria della Pagina e Mappatura Tipografica . . . 22
8.3 Architettura di Paginazione Deterministica (≥ 2 Pagine) . . . 22
8.4 Algoritmo di Paginazione e Word-Wrapping Dinamico . . . 22
8.5 Sandboxing e FileProvider Android . . . 23

9 Sottosistema di Backup, Serializzazione e Merge **23**
9.1 Schema Canonico SIMO JSON v2 . . . 23
9.2 Algoritmo di Risoluzione Conflitti a Due Fasi . . . 24

10 Testing, Validazione e Qualità del Codice **25**
10.1 Pipeline di Distribuzione, Firma Crittografica e Keystore Personale . . . 25

11 Conclusioni e Scalabilità Futura **27**
11.1 Valutazione dei Risultati . . . 27
11.2 Miglioramenti Futuri . . . 27

1 Introduzione

1.1 Problem Statement & Contesto di Dominio

Nei moderni processi di preparazione culinaria professionale e di alta pasticceria, la gestione delle ricette presenta una complessità computazionale e strutturale che eccede ampiamente le capacità di un semplice archivio testuale o tabellare.

I requisiti critici individuati sul campo evidenziano le seguenti problematiche:

1. **Gerarchie Multi-Livello (Composite Recipes):** Le preparazioni complesse non sono insiemi piatti di materie prime, bensì aggregati di preparazioni intermedie (*sotto-ricette*). Ad esempio, un *Cornetto alla Crema* include una pasta sfogliata lievitata e una *Crema Pasticcera*. Qualsiasi variazione della resa o della ricetta base della crema deve propagarsi in modo deterministico e reattivo all'intero albero di composizione;

2. **Inversione di Scala da Vincolo di Dispensa (Reverse Scaling):** In ambiente di laboratorio o domestico, la quantità di resa finale è frequentemente vincolata dalla disponibilità residua di un ingrediente critico (esempio: 120 g di burro anziché 175 g). Il sistema deve essere in grado di calcolare il fattore di scala inverso $k \in \mathbb{R}^+$ rispetto a un generico nodo dell'albero ed estendere la proporzione all'intero grafo;

3. **Tracciabilità Normativa degli Allergeni (Regolamento UE 1169/2011):** L'obbligo di legge impone l'individuazione e la segnalazione dei 14 allergeni normativi. La presenza di un allergene in una foglia del grafo deve essere propagata ricorsivamente fino alla radice;

4. **Food Cost Contabile in Tempo Reale:** Necessità di quantificare all'istante l'incidenza economica di ciascun lotto di produzione e il costo industriale di ogni singola porzione, considerando conversioni tra unità di massa (g, kg), di volume (ml, l) e unità discrete quantizzate $(pezzi; unità)$;

5. **Operatività Senza Soluzione di Continuità (Cooking Mode):** Esigenza di un'interfaccia esecutiva durante la cottura che mantenga il display costantemente attivo, gestisca timer asincroni multipli con allarmi acustici e vibrazione, e supporti aggiunte rapide di tempo non intrusive;

6. **Privacy e Sicurezza delle Chiavi API di Terze Parti:** Protezione crittografica hardware-backed contro attacchi di estrazione locale o perdite accidentali nei backup per le chiavi di accesso a modelli generativi remoti (Google Gemini e OpenAI ChatGPT);

7. **Variazione di Massa Termica in Cottura (Resa Termica & Calo Peso):** Nei trattamenti termici (cottura in forno, padella, bollitura, frittura) gli alimenti subiscono alterazioni fisiche rilevanti della massa: calo peso per evaporazione dell'acqua (es. torte, arrosti $-10\% \div -20\%$) oppure aumento di massa per assorbimento di liquidi idratanti (es. pasta secca e riso $+100\% \div +150\%$). L'architettura deve modellare tale fenomeno termodinamico calcolando la resa cotta finita a porzione e la conseguente variazione di densità dei nutrienti;

8. **Dichiarazione Nutrizionale Ufficiale (Regolamento UE n. 1169/2011):** Necessità di determinare in modo scientifico la concentrazione o diluizione post-cottura dei 6 nutrienti normativi (Energia in kJ/kcal, Grassi, Carboidrati, Proteine, Acqua, Sodio e Sale equivalente), fornendo la doppia rappresentazione analitica *Per 100g* e *Per Porzione* contrassegnata dalla dicitura legale di stima.

1.2 Requisiti Non Funzionali e Vincoli Architetturali

Il sistema è stato ingegnerizzato rispettando i seguenti vincoli prestazionali e architetturali:

- **Offline-First:** Tutte le funzioni analitiche, di scaling, di navigazione del grafo, di food cost e di cottura guidata devono operare a latenza zero anche in assenza totale di connettività di rete;
- **Tempo di Calcolo dello Scaling:** Complessità temporale $O(|V| + |E|)$ rispetto al grafo di composizione, con tempo di re-rendering $\leq 16\,\text{ms}$ a $60\,\text{FPS}$ per evitare frame-drop nell'interfaccia utente dichiarativa;
- **Assenza di Dipendenze Circolari:** Garanzia formale che il grafo delle ricette rimanga in ogni istante un **Grafo Diretto Aciclico** (**DAG**);
- **Memory Footprint Controllato:** Nessuna allocazione ridondante di bitmap durante l'esportazione PDF o la gestione dei timer; rilascio immediato delle risorse audio dell'Hardware Abstraction Layer.

2 Architettura del Software e Pattern Architetturali

SIMO adotta una combinazione rigorosa di **Clean Architecture** e **Model-View-ViewModel** (**MVVM**), orchestrata mediante il pattern **Unidirectional Data Flow** (**UDF**) nativo del framework Jetpack Compose.



Figura 1: Stratificazione architetturale ad accoppiamento debole di SIMO.

2.1 Unidirectional Data Flow (UDF) e State Immutability

Ogni schermata applicativa è funzione pura dello stato corrente (*State-Driven UI*):

$$\text{UI} = f(\text{UiState})$$

I ViewModel non espongono variabili mutabili pubbliche: ogni transizione di stato avviene tramite istanze di data class immutabili incapsulate in `StateFlow<T>`.

```
val uiState: StateFlow<RecipeListUiState> = combine(
    repository.allRecipes,
    _filterState
) { recipes, filter ->
    // Calcolo filtri, ordinamento e ricerca su pool di background
    val sortedAvailableTags = RecipeTags.getAllSortedTags(recipes.map { it.
    category })
    val byTags = if (filter.selectedTags.isEmpty()) recipes
                 else recipes.filter { r -> filter.selectedTags.any { it.equals(
    r.category, true) } }
    val byFavorites = if (filter.favoritesOnly) byTags.filter { it.isFavorite }
    else byTags
    val filtered = if (filter.query.isBlank()) byFavorites
                   else byFavorites.filter { it.name.contains(filter.query, true
    ) }
```

```
12
13
14
15      val sorted = when (filter.sort) {
16          RecipeSortOption.NAME_ASC -> filtered.sortedWith(compareBy(String.
        CASE_INSENSITIVE_ORDER) { it.name })
17          RecipeSortOption.NAME_DESC -> filtered.sortedWith(compareByDescending(
        String.CASE_INSENSITIVE_ORDER) { it.name })
18          RecipeSortOption.DATE_DESC -> filtered.sortedByDescending { it.createdAt
         }
19          RecipeSortOption.DATE_ASC -> filtered.sortedBy { it.createdAt }
20      }
21      RecipeListUiState(recipes = sorted, availableTags = sortedAvailableTags,
        ...)
22  }.flowOn(Dispatchers.Default)
23   .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000),
        RecipeListUiState())
```

Listing 1: Esempio di emissione di stato reattivo in RecipeListViewModel

2.2 Concorrenza e Thread Safety

L'applicazione fa uso sistematico delle Kotlin Coroutines, assegnando rigorosamente i contesti di esecuzione:

- `Dispatchers.Main`: Riservato esclusivamente al rendering dichiarativo di Jetpack Compose e al binding degli eventi UI;
- `Dispatchers.Default`: Dedicato al calcolo analitico intensivo (espansione del DAG, determinazione dei percorsi, ordinamento, estrazione regex dei timer e ricalcolo proporzionale);
- `Dispatchers.IO`: Adibito all'I/O su database Room SQLite, operazioni di crittografia hardware, chiamate REST HTTP di Gemini e stream di scrittura PDF/JSON.

2.3 Lifecycle Optimization: Warm-up Asincrono e Pre-caching all'Avvio

Per prevenire l'anti-pattern del *Flash of Empty State* (in cui la schermata home mostra un layout vuoto per alcune centinaia di millisecondi durante l'interrogazione asincrona iniziale del database), l'architettura implementa una fase di warm-up reattivo. Durante lo splash iniziale di avvio, il repository esegue il pre-caricamento immediato della collezione completa delle entità `RecipeWithComponents`. L'emissione iniziale del relativo `StateFlow` avviene in modo concorrente con la composizione della UI, garantendo che al passaggio alla schermata del ricettario l'elenco sia già interamente popolato e ordinato, azzerando qualsiasi percezione di caricamento differito.

3 Domain Engine: Teoria dei Grafi e Calcolo Matematico

Il nucleo algoritmico risiede nel file `MathEngine.kt`, implementato come modulo Kotlin puro privo di dipendenze verso il framework Android.

3.1 Formalizzazione del Ricettario come DAG Pesato

Definizione 1 (Grafo delle Ricette)**.** *Sia $G = (V; E; W)$ un grafo orientato pesato finito dove:*

- $V = V_{raw} \cup V_{recipe}$ *è l'insieme dei vertici, partizionato in materie prime V_{raw} e preparazioni composte V_{recipe}.*
- $E \subseteq V_{recipe} \times V$ *è l'insieme degli archi orientati $(u; v)$, indicante che la ricetta u richiede l'ingrediente o sotto-ricetta v.*
- $W : E \to \mathbb{R}^+ \times \mathcal{U}$ *è la funzione peso che associa ad ogni arco la tupla $(q; u)$ rappresentante la quantità base q e l'unità di misura $u \in \mathcal{U}$.*

Affinché il calcolo delle dosi converga in tempo finito e non generi paradossi fisici (ad esempio: la Crema Pasticcera che richiede Cornetti che a loro volta richiedono Crema Pasticcera), G deve soddisfare il vincolo fondamentale di aciclicità:

$$\forall v \in V, \quad \nexists \text{ cammino orientato } v \rightsquigarrow v \quad \iff \quad G \text{ è un DAG}$$

3.2 Algoritmo di Rilevamento Cicli Preventivo (`wouldCreateCycle`)

Prima di consentire l'inserimento o la modifica di un arco $(u; v)$ dove u è la ricetta genitore e v è la ricetta figlia candidata, il sistema valuta se esiste un cammino diretto $v \rightsquigarrow u$.

Teorema 1 (Condizione di Aciclicità su Inserimento di Arco)**.** *Dato un DAG $G = (V; E)$ e un arco candidato $(u; v)$, il nuovo grafo $G' = (V; E \cup \{(u; v)\})$ è un DAG se e solo se u non è raggiungibile da v in G.*

Dimostrazione. Se u fosse raggiungibile da v, esisterebbe un cammino orientato $p = (v; w_1; w_2; \ldots; w_k; u)$ in G. Aggiungendo l'arco $(u; v)$, si formerebbe il ciclo $C = (u; v; w_1; \ldots; w_k; u)$, violando la proprietà di DAG. Se u non è raggiungibile da v, nessun nuovo cammino originante da u potrà mai ricongiungersi a un suo predecessore, preservando l'aciclicità. □

L'implementazione in `MathEngine.kt` impiega una Breadth-First Search (BFS) ottimizzata:

```
fun wouldCreateCycle(
    parentRecipeId: String,
    candidateChildId: String,
    existingEdges: List<Pair<String, String>>
): Boolean {
    if (parentRecipeId.isBlank() || candidateChildId.isBlank()) return false
    if (parentRecipeId == candidateChildId) return true // Auto-loop immediato

    // Costruzione lista di adiacenza
    val adjacency = mutableMapOf<String, MutableList<String>>()
    for ((parent, child) in existingEdges) {
        if (parent.isNotBlank() && child.isNotBlank()) {
            adjacency.getOrPut(parent) { mutableListOf() }.add(child)
        }
    }

    val queue = ArrayDeque<String>()
    queue.add(candidateChildId)
    val visited = mutableSetOf<String>()
```

```
20
21      while (queue.isNotEmpty()) {
22          val curr = queue.removeFirst()
23          if (curr == parentRecipeId) return true // Ciclo rilevato!
24          if (visited.add(curr)) {
25              adjacency[curr]?.forEach { next ->
26                  if (!visited.contains(next)) queue.add(next)
27              }
28          }
29      }
30      return false
31  }
```

Listing 2: Algoritmo BFS per la validazione aciclica di nuovi componenti

Complessità Computazionale:

- **Tempo:** $O(|V| + |E|)$ nel caso peggiore (visita di tutti i vertici e archi raggiungibili).
- **Spazio:** $O(|V|)$ per l'allocazione della coda e dell'insieme di visitati.

3.3 Algoritmo di Espansione ad Albero Scalato (`buildScaledTree`)

Dato un nodo radice $R \in V_{\text{recipe}}$ e un target di resa $Y_{\text{target}} \in \mathbb{R}^+$, il fattore di scala primario k_R è calcolato come:

$$k_R = \frac{Y_{\text{target}}}{Y_{\text{base}}(R)}$$

Per ciascun componente $c = (R, i)$ con quantità base $q_{\text{base}}(c)$:

$$q_{\text{scaled}}(c) = q_{\text{base}}(c) \cdot k_R$$

Se il componente i è a sua volta una ricetta $i \in V_{\text{recipe}}$, essa richiede una quantità pari a $q_{\text{scaled}}(c)$ espressa nell'unità di resa della sotto-ricetta. Di conseguenza, il fattore di scala per la sotto-ricetta k_i diventa:

$$k_i = \frac{q_{\text{scaled}}(c)}{Y_{\text{base}}(i)} = \frac{q_{\text{base}}(c) \cdot k_R}{Y_{\text{base}}(i)}$$

Il processo prosegue ricorsivamente fino alle foglie del grafo (V_{raw}).

Protezione da Stack Overflow su Ricorsione Profonda

Per garantire la robustezza del runtime Android e prevenire esaurimenti di stack su grafi patologici, l'algoritmo impone una soglia massima di guardia `MAX_RECURSION_DEPTH = 25` e mantiene un set di controllo `visitedRecipeIds` ad ogni ramo dell'albero.

3.4 Modello di Reverse Scaling

Il Reverse Scaling consente di calcolare il moltiplicatore k^* a partire da un generico ingrediente j per cui l'operatore dispone di una quantità limite $q_{\text{disp}}(j)$:

$$k^* = \frac{q_{\text{disp}}(j)}{q_{\text{base}}(j)}$$

Tutti gli altri componenti $m \neq j$ e la resa complessiva Y vengono immediatamente riparametrati:

$$Y^* = Y_{\text{base}} \cdot k^*, \qquad \forall m \in \text{Componenti}, \quad q^*(m) = q_{\text{base}}(m) \cdot k^*$$

3.5 Quantizzazione e Sistema delle Unità di Misura

In gastronomia e pasticceria non tutte le grandezze possono essere trattate come numeri reali continui in $\mathbb{R}^+$. L'architettura classifica le unità $\mathcal{U}$ in tre categorie ortogonali:

1. **Unità Continue Ponderali e Volumetriche (g, kg, ml, l):** I valori scalati mantengono fino a due cifre decimali se $q < 100$, oppure vengono approssimati al numero intero se $q \geq 100$ per uniformità con le bilance digitali professionali.

2. **Unità Discrete Quantizzate ($pezzi; unità$):**

$$\text{roundIntegerAmount}(q) = \max(1; \lfloor q + 0.5 \rfloor)$$

   Nel caso specifico delle uova intere o dei tuorli, un valore come 2.4 unità non è utilizzabile direttamente senza creare sprechi. La quantizzazione arrotonda all'intero più prossimo salvaguardando il minimo di 1 unità.

3. **Quantificatori Qualitativi ($q.b.$):** Non sono soggetti a moltiplicazione scalare e mantengono costantemente la dicitura letterale "q.b.".

3.6 Modello Analitico del Food Cost

Sia $P(i)$ il prezzo per unità di misura associato all'ingrediente i. Il costo elementare $C(c)$ del componente scalato c con quantità $q(c)$ e unità u_r, dato un prezzo p con unità u_p, è formalizzato come:

$$C(c) = q(c) \cdot \mu(u_r; u_p) \cdot p$$

Dove $\mu(u_r, u_p)$ è l'operatore di conversione metrica:

$$\mu(u_r; u_p) = \begin{cases} 1 & \text{se } u_r = u_p \vee (u_r \in \{g, ml\} \wedge u_p \in \{g, ml\}) \vee (u_r \in \{kg, l\} \wedge u_p \in \{kg, l\}) \\ 10^{-3} & \text{se } u_r \in \{g, ml\} \wedge u_p \in \{kg, l\} \\ 10^{3} & \text{se } u_r \in \{kg, l\} \wedge u_p \in \{g, ml\} \\ \frac{\text{gramsPerUnit}}{1000} & \text{se } u_r \in \{\text{unità, pz}\} \wedge u_p = kg \\ \text{gramsPerUnit} & \text{se } u_r \in \{\text{unità, pz}\} \wedge u_p = g \\ 0 & \text{se } u_r = \text{q.b.} \end{cases}$$

Nel contesto della ricetta, un ingrediente la cui dose è qualificata come $q.b.$ (quanto basta) presenta $C(c) = 0$, trattandosi di un dosaggio non ponderale a discrezione dello chef (ad es. un pizzico di sale o spezie) che non deve alterare artificiosamente il costo base del lotto di produzione.

Il costo totale dell'impasto C_{totale} e il costo unitario per porzione C_{porzione} si ottengono mediante accumulazione ricorsiva bottom-up:

$$C_{\text{totale}} = \sum_{c \in \text{Foglie}} C(c) + \sum_{s \in \text{Sotto-Ricette}} C_{\text{totale}}(s)$$

$$C_{\text{porzione}} = \frac{C_{\text{totale}}}{Y_{\text{target}}}$$

3.7 Coerenza Metrica tra Food Cost e Carrello della Spesa

L'aggregazione degli ingredienti nella lista della spesa adotta l'identico operatore di conversione metrica $\mu(u_r, u_p)$. In questo modo, un ingrediente con costo espresso al chilogrammo (es. €1.80/kg) la cui ricetta richiede 250 g produce nel carrello della spesa una stima pari a:

$$\text{Costo}_{\text{spesa}} = 250 \cdot 10^{-3} \cdot 1.80 = 0.45\,€$$

eliminando qualsiasi discrepanza tra il Food Cost stimato nel dettaglio ricetta e la spesa complessiva.

Per gli ingredienti a quantitativo qualitativo indeterminato (*q.b.*), la lista della spesa eredita l'unità di misura associata al prezzo d'acquisto (u_p) e adotta una quantità deterministica standard per consentire l'approvvigionamento del formato commerciale minimo:

$$q_{\text{spesa}} = \begin{cases} 100 & \text{se } u_p \in \{\text{g}, \text{ml}\} \\ 1 & \text{per unità discrete o formati commerciali (pz, unità)} \end{cases}$$

fornendo all'utente un punto di partenza facilmente calibrabile direttamente dalla checklist d'acquisto.

3.8 Tipografia e Algoritmo di Line-Breaking Antifrattura

Nella presentazione delle stringhe descrittive complesse degli ingredienti (specialmente denominazioni protette quali *Formaggio Parmigiano Reggiano DOP* o *Olio Extravergine d'Oliva*), l'algoritmo di rendering di Jetpack Compose è stato vincolato per inibire la sillabazione infrasillabica (`Hyphens.None`) e forzare il wrapping esclusivamente sui delimitatori di parola (`LineBreak.Paragraph`). Ciò previene la frammentazione antiestetica dei lemmi merceologici sui display a densità variabile.

3.9 Stabilità Tipografica dei Badge Allergeni e Prevenzione della Compressione Orizzontale

Nella scheda di riepilogo normativo degli allergeni (`RecipeAllergensCard`), ciascuna voce associa la denominazione merceologica (es. *Anidride solforosa e solfiti*) al rispettivo identificatore normativo UE (es. *Allergene #12*). A causa del comportamento di misurazione unconstrained dei layout `Row` in Compose con allineamento `SpaceBetween`, stringhe particolarmente estese provocavano una contrazione dell'area utile assegnata al badge laterale, inducendo il wrapping anomalo della sequenza (*"Aller" / "gene" / "#12"*).

La correzione architetturale applica due vincoli concorrenti di layout:

1. Il componente testuale della denominazione riceve il modificatore flessibile non saturante:

```
Text(
    text = allergen.displayName,
    modifier = Modifier.weight(1f, fill = false)
)

```

Listing 3: Vincolo dimensionale flessibile per la label dell'allergene

   Ciò impone a Compose di misurare prioritariamente il badge laterale alla sua dimensione intrinseca ideale, assegnando alla denominazione lo spazio residuo senza esercitare pressione di restringimento.

2. Il badge numerico è blindato con vincoli di linea singola inderogabili:

```
Text(
    text = "Allergene #${allergen.ordinal + 1}",
    maxLines = 1,
    softWrap = false
)

```

Listing 4: Inibizione del wrapping sul badge identificativo

Tale configurazione garantisce che su qualsiasi densità dpi e con qualsivoglia fattore di scala dei caratteri di sistema, il contrassegno normativo si presenti sempre integro su un'unica riga.

3.10 Equivalenza Culinaria Massa-Volume (1 : 1) nel Food Cost e nella Spesa

Nell'esperienza gastronomica quotidiana, molti ingredienti fluidi o semi-densi (acqua, latte, succhi, infusi, brodi, caffè, panna liquida, ecc.) vengono misurati indifferentemente in volume (ml, l) o in peso (g, kg). L'operatore di conversione $\mu(u_r; u_p)$ integra il principio di **equivalenza culinaria isotropa**:

$$1\,\mathrm{g} \equiv 1\,\mathrm{ml}, \qquad 1\,\mathrm{kg} \equiv 1\,\mathrm{l}$$

In questo modo, se un ingrediente viene dosato in millilitri (es. 300 ml di latte o caffè) ma il listino d'acquisto è espresso al chilogrammo (es. €25.00/kg), il costo elementare non genera anomalie di scala né moltiplicazioni improprie:

$$C = 300\,\mathrm{ml} \cdot 10^{-3} \cdot 25.00 = 7.50\,€$$

L'aggregatore della lista della spesa applica la medesima equivalenza, garantendo coerenza assoluta tra magazzino, preventivo economico e quantità da acquistare.

3.11 Modello Matematico della Variazione di Massa in Cottura (Resa Termica)

Durante la trasformazione termica, la massa complessiva della preparazione si modifica in funzione del parametro `cookingWeightModifier` ($\Delta M \in [-100, +\infty)$):

Definizione 2 (Massa Cruda e Massa Cotta Finita)**.** *Sia W_{raw} la massa totale dell'impasto a crudo, calcolata come sommatoria dei pesi di tutti gli ingredienti ponderali (g, kg) o convertibili (ml, l):*

$$W_{raw} = \sum_{c \in Foglie} q_{scaled}(c) \cdot \rho(u_c)$$

dove $\rho(u_c) = 1$ per grammi e millilitri, $\rho(u_c) = 1000$ per chilogrammi e litri, e $\rho(u_c) = gramsPerUnit$ per unità discrete. La massa cotta finita W_{cooked} è definita come:

$$W_{cooked} = W_{raw} \cdot \left(1 + \frac{\Delta M}{100}\right)$$

Si distinguono tre regimi termodinamici:

- **Calo Peso Termico ($\Delta M < 0$):** Evaporazione d'acqua per torte da forno, crostate, panificati e arrosti (es. $\Delta M = -10\% \implies W_{\text{cooked}} = 0.90 \cdot W_{\text{raw}}$);
- **Idratazione e Aumento di Massa ($\Delta M > 0$):** Assorbimento di brodo o acqua di cottura da parte di amidi e cereali secchi (es. pasta secca $\Delta M = +120\% \implies W_{\text{cooked}} = 2.20 \cdot W_{\text{raw}}$);
- **Regime Neutro ($\Delta M = 0$):** Preparazioni fredde, salse crude, cocktail o gelati dove non vi è perdita né guadagno di solvente volatile.

Il peso finito cotto di ciascuna porzione è dato da:

$$w_{\text{portion, cooked}} = \frac{W_{\text{cooked}}}{N_{\text{portions}}}$$

3.12 Modello di Calcolo della Dichiarazione Nutrizionale UE (Regolamento UE n. 1169/2011)

Ciascuna materia prima $i \in V_{\text{raw}}$ possiede un vettore nutrizionale intrinseco a crudo riferito a 100 g:

$$\mathbf{Nut}_i = \begin{pmatrix} E_i \\ C_i \\ F_i \\ P_i \\ W_i \\ S_i \end{pmatrix} = \begin{pmatrix} \text{Energia (kcal)} \\ \text{Carboidrati (g)} \\ \text{Grassi (g)} \\ \text{Proteine (g)} \\ \text{Acqua/Umidità (g)} \\ \text{Sodio (mg)} \end{pmatrix}$$

L'Energia in kiloJoule viene calcolata secondo il fattore di conversione standard UE:

$$E_{\text{kJ}} = E_{\text{kcal}} \times 4.184$$

La quantità equivalente di **Sale (g)** è formalizzata dal Regolamento UE 1169/2011 moltiplicando il Sodio (espresso in grammi) per la costante stechiometrica del cloruro di sodio (2.5):

$$\text{Sale (g)} = \frac{Sodio_{\text{mg}}}{1000} \times 2.5$$

La quantità assoluta totale di ciascun nutriente nella ricetta prima della cottura è:

$$\mathbf{Nut}_{\text{tot}} = \sum_{c \in \text{Componenti}} \frac{q_{\text{grams}}(c)}{100} \cdot \mathbf{Nut}_c$$

3.13 Algoritmo di Concentrazione e Diluizione Nutrizionale Post-Cottura

Teorema 2 (Fattore di Concentrazione Nutrizionale sulla Resa Cotta)**.** *Dato che i macronutrienti (grassi, proteine, carboidrati) e i micronutrienti minerali (sodio/sale) non subiscono evaporazione a temperature di cottura ordinarie ($\leq 250°C$), la massa assoluta dei nutrienti si conserva:* $\mathbf{Nut}_{tot,\ cooked} = \mathbf{Nut}_{tot,\ raw}$*. Pertanto, la concentrazione di ciascun nutriente per* 100 *g di prodotto finito cotto è funzione inversa della massa cotta:*

$$\mathbf{Nut}^{100g}_{cooked} = \frac{\mathbf{Nut}_{tot}}{W_{cooked}} \times 100 = \frac{\mathbf{Nut}_{tot}}{W_{raw} \cdot \left(1 + \frac{\Delta M}{100}\right)} \times 100$$

Dimostrazione. Se $\Delta M < 0$ (evaporazione dell'acqua), il denominatore decresce ($W_{\text{cooked}} < W_{\text{raw}}$), determinando un aumento della densità calorica e nutrizionale per 100 grammi (effetto di concentrazione). Viceversa, se $\Delta M > 0$ (assorbimento d'acqua), il denominatore cresce, inducendo una diluizione della densità nutrizionale per 100 grammi. In entrambi i casi, l'apporto nutrizionale **per porzione** rimane invariato rispetto al totale:

$$\mathbf{Nut}_{\text{portion}} = \frac{\mathbf{Nut}_{\text{tot}}}{N_{\text{portions}}}$$

□

3.14 Regole di Conservazione Gastronomica dell'Unità di Resa

Per prevenire conversioni inappropriate operate dai modelli linguistici (ad es. forzare a "porzioni" preparazioni con resa unitaria intrinseca), il motore impone le seguenti regole deterministiche:

- Se l'unità inserita dall'utente appartiene all'insieme dei descrittori ammissibili per la classe di ricetta (es. *torta, crostata* per categoria Dolci; *bicchieri, calici, tazze* per Caffetteria o Drink; *panini, burger* per la categoria Panini; *teglia, pizza* per Pizze), il generatore AI è vincolato contrattualmente a preservare fedelmente tale unità;
- Solo in caso di incompatibilità semantica manifesta (es. *bicchieri* per un arrosto di carne o per un risotto), il sistema applica il fallback sicuro al formato generico "porzioni".

4 Strato Dati, Room Database e Persistenza Reattiva

4.1 Schema Relazionale e Normalizzazione

Il database relazionale locale è implementato mediante **Android Room** (schema version 12), configurato senza esportazione di schema per ottimizzazione di build KSP.

```
TABLE items (id TEXT PK, name TEXT, type TEXT, defaultUnit TEXT, pricePerUnit REAL, priceUnit
TEXT, gramsPerUnit REAL, allergens TEXT, caloriesKcal REAL NOT NULL DEFAULT 0.0, carbsG REAL
NOT NULL DEFAULT 0.0, fatsG REAL NOT NULL DEFAULT 0.0, proteinsG REAL NOT NULL DEFAULT 0.0,
waterG REAL NOT NULL DEFAULT 0.0, sodiumMg REAL NOT NULL DEFAULT 0.0)

TABLE recipes (id TEXT PK, name TEXT, description TEXT, baseYieldQuantity REAL, yieldUnit TEXT,
instructions TEXT, isDefault INTEGER, isFavorite INTEGER, category TEXT, createdAt INTEGER,
cookingWeightModifier REAL NOT NULL DEFAULT 0.0)

TABLE recipe_components (id INTEGER PK AUTOINCREMENT, parentRecipeId TEXT, childItemId TEXT,
baseQuantity REAL, unit TEXT, sortOrder INTEGER)

TABLE shopping_items (id INTEGER PK AUTOINCREMENT, itemName TEXT, quantity REAL, unit TEXT,
unitPrice REAL NOT NULL DEFAULT 0.0, priceUnit TEXT, source TEXT, isChecked INTEGER, createdAt
INTEGER)
```

Figura 2: Schema DDL delle entità relazionali in Room.

4.2 Polimorfismo Relazionale degli Item

Per supportare la composizione arbitraria del DAG, l'entità `ItemEntity` è polimorfa:

- Se `type = "RAW_INGREDIENT"`, l'elemento rappresenta una materia prima fisica primaria (esempio: *Latte intero*, *Farina 00*).
- Se `type = "RECIPE"`, l'elemento funge da proxy referenziabile verso una tupla nella tabella `recipes` avente la medesima chiave primaria (`items.id = recipes.id`).

4.3 Prevenzione dei "Dangling Pointers" Relazionali

Uno dei punti più delicati dell'architettura riguarda l'eliminazione di una ricetta utilizzata come ingrediente in altre preparazioni.

Politica di Eliminazione Sicura

Se l'utente elimina la ricetta R, ma R è utilizzata in $P_1, P_2, \ldots, P_n$:

1. La tupla corrispondente viene cancellata dalla tabella `recipes`;
2. I suoi componenti interni come genitore vengono rimossi da `recipe_components`;
3. **L'entità in `items` non viene eliminata**, bensì convertita atomicamente da `type = "RECIPE"` a `type = "RAW_INGREDIENT"`.

In questo modo, le ricette genitori $P_1, \ldots, P_n$ continuano a funzionare regolarmente considerando R come una materia prima acquistabile/pre-composta, senza generare eccezioni di puntatore nullo o corruzioni nel grafo!

```
1 suspend fun deleteRecipe(recipeId: String, convertToRaw: Boolean = false):
      DeleteResult {
2     return try {
3         val referencing = recipeDao.getParentRecipeNamesUsingChild(recipeId)
4         database.withTransaction {
```

```
            recipeDao.deleteComponentsForRecipe(recipeId)
            recipeDao.deleteRecipeById(recipeId)
            if (convertToRaw || referencing.isNotEmpty()) {
                itemDao.updateItemType(recipeId, "RAW_INGREDIENT")
            } else {
                itemDao.deleteItemById(recipeId)
            }
        }
        DeleteResult.Success
    } catch (e: Exception) {
        DeleteResult.Error(e.message ?: "Errore eliminazione")
    }
}
```

Listing 5: Transazione atomica di eliminazione con conversione in RecipeRepository

4.4 Evoluzione dello Schema e Migrazioni Database

La migrazione incrementale alla versione 10 aggiunge la colonna `allergens` valorizzando retroattivamente le materie prime base, mentre l'evoluzione verso la versione 12 introduce il supporto economico e di costo unitario per gli articoli della spesa:

```
val MIGRATION_9_10 = object : Migration(9, 10) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL("ALTER TABLE items ADD COLUMN allergens TEXT NOT NULL DEFAULT
     ''")
        db.execSQL("UPDATE items SET allergens = 'LATTE' WHERE id = 'ing_latte'"
    )
        db.execSQL("UPDATE items SET allergens = 'UOVA' WHERE id = 'ing_tuorli'"
    )
        db.execSQL("UPDATE items SET allergens = 'GLUTINE' WHERE id = '
    ing_farina_manitoba'")
        db.execSQL("UPDATE items SET allergens = 'UOVA,LATTE' WHERE id = '
    recipe_crema_pasticcera'")
    }
}

val MIGRATION_11_12 = object : Migration(11, 12) {
    override fun migrate(db: SupportSQLiteDatabase) {
        // Supporto nutrizionale esteso a 6 grandezze sulle materie prime
        db.execSQL("ALTER TABLE items ADD COLUMN caloriesKcal REAL NOT NULL
    DEFAULT 0.0")
        db.execSQL("ALTER TABLE items ADD COLUMN carbsG REAL NOT NULL DEFAULT
    0.0")
        db.execSQL("ALTER TABLE items ADD COLUMN fatsG REAL NOT NULL DEFAULT 0.0
    ")
        db.execSQL("ALTER TABLE items ADD COLUMN proteinsG REAL NOT NULL DEFAULT
     0.0")
        db.execSQL("ALTER TABLE items ADD COLUMN waterG REAL NOT NULL DEFAULT
    0.0")
        db.execSQL("ALTER TABLE items ADD COLUMN sodiumMg REAL NOT NULL DEFAULT
    0.0")

        // Supporto termico e calo peso in cottura sulla ricetta
        db.execSQL("ALTER TABLE recipes ADD COLUMN cookingWeightModifier REAL
    NOT NULL DEFAULT 0.0")
    }
}
```

Listing 6: Dichiarazione migrazioni SQL in AppDatabase

4.5 Pattern di Modifica Reattiva: `EditableComponent`

Nella gestione dello stato di editing delle preparazioni, `RecipeEditViewModel` adotta la struttura immutabile `EditableComponent`:

```
data class EditableComponent(
    val tempId: String = UUID.randomUUID().toString(),
    val itemId: String,
    val itemName: String,
    val itemType: ItemType,
    val quantity: String,
    val unit: String,
    val pricePerUnit: Double? = null,
    val priceUnit: String? = null,
    val allergens: List<Allergen> = emptyList(),
    val caloriesKcal: Double = 0.0,
    val carbsG: Double = 0.0,
    val fatsG: Double = 0.0,
    val proteinsG: Double = 0.0,
    val waterG: Double = 0.0,
    val sodiumMg: Double = 0.0
)
```

Listing 7: Data model reattivo per la modifica di ingredienti e materie prime

Il metodo atomico `updateComponentDetails(...)` applica una sincronizzazione bidirezionale:

1. Aggiorna in modo immutabile l'elenco dei componenti della ricetta nello stato UI;
2. Se il componente è una materia prima fisica (`itemType != RECIPE`), esegue una coroutine asincrona su `Dispatchers.IO` persistendo nel database Room le modifiche a nome, unità predefinita, costo unitario e allergeni all'interno della tabella `items`.

Ciò assicura che qualunque rettifica apportata dall'utente o dall'assistente AI Gemini persista globalmente nell'anagrafica del ricettario.

4.6 Algoritmo di Riordinamento Reattivo Continuo (Drag & Drop) e Spostamento Discreto

Nel modulo di composizione della ricetta (`RecipeEditScreen`), l'ordinamento sequenziale degli ingredienti risponde a due modalità operative:

1. **Spostamento Discreto a Passo Singolo:** I metodi `moveComponentUp(index)` e `moveComponentDown(ind` operano scambi puntuali tra indici adiacenti su lista immutabile (`toMutableList().apply { Collections.swap(...) }`).
2. **Trascinamento Continuo Multi-Posizione (Fluid Drag & Drop):** L'implementazione convenzionale dei pointer gesture detector in Compose (`detectDragGesturesAfterLongPress`) acquisisce per chiusura lessicale (*closure capture*) l'indice dell'elemento all'avvio del tocco; ciò comportava l'arresto dello scorrimento dopo un singolo scambio.

Per consentire lo scorrimento continuo dall'inizio alla fine dell'elenco senza mai sollevare il dito, è stato implementato un automa dinamico a stato reattivo:

```
var activeDragId by remember { mutableStateOf<String?>(null) }
var dragOffsetY by remember { mutableFloatStateOf(0f) }
val itemSlotHeightPx = with(LocalDensity.current) { 80.dp.toPx() }

// All'interno di pointerInput(comp.tempId):
detectDragGesturesAfterLongPress(
    onDragStart = { activeDragId = comp.tempId; dragOffsetY = 0f },
```

```
    onDrag = { _, dragAmount ->
        dragOffsetY += dragAmount.y
        // Spostamento verso il basso (multi-slot)
        while (dragOffsetY > itemSlotHeightPx) {
            val list = viewModel.uiState.value.components
            val curIdx = list.indexOfFirst { it.tempId == activeDragId }
            if (curIdx != -1 && curIdx < list.size - 1) {
                viewModel.moveComponent(curIdx, curIdx + 1)
                dragOffsetY -= itemSlotHeightPx
            } else break
        }
        // Spostamento verso l'alto (multi-slot)
        while (dragOffsetY < -itemSlotHeightPx) {
            val list = viewModel.uiState.value.components
            val curIdx = list.indexOfFirst { it.tempId == activeDragId }
            if (curIdx != -1 && curIdx > 0) {
                viewModel.moveComponent(curIdx, curIdx - 1)
                dragOffsetY += itemSlotHeightPx
            } else break
        }
    },
    onDragEnd = { activeDragId = null; dragOffsetY = 0f },
    onDragCancel = { activeDragId = null; dragOffsetY = 0f }
)
```

Listing 8: Algoritmo continuo multi-slot basato su ID dinamico e loop iterativo

L'algoritmo calcola in tempo reale la posizione corrente dell'`activeDragId` nello StateFlow immutabile, decrementando o incrementando iterativamente l'accumulatore `dragOffsetY` in quanti multipli dell'altezza di slot (`itemSlotHeightPx`). L'esperienza visiva è resa continua tramite accelerazione hardware con `Modifier.graphicsLayer { translationY = dragOffsetY; scaleX = 1.02f; scaleY = 1.02f }` ed elevazione su layer prioritario con `Modifier.zIndex(10f)`.

5 Sicurezza Hardware-Backed e Crittografia KeyStore

La memorizzazione delle chiavi API per servizi cloud generativi remoti (come Google Gemini) rappresenta una delle principali superfici di attacco nei sistemi mobile. Memorizzare token in chiaro nelle `SharedPreferences` o nei file XML espone al furto dei crediti tramite decompilazione APK, backup non cifrati o ispezione del file system root.

5.1 Architettura del Modulo `CryptoUtils`

SIMO implementa una soluzione di crittografia hardware-backed a livello TEE (*Trusted Execution Environment*) o StrongBox Keymaster.

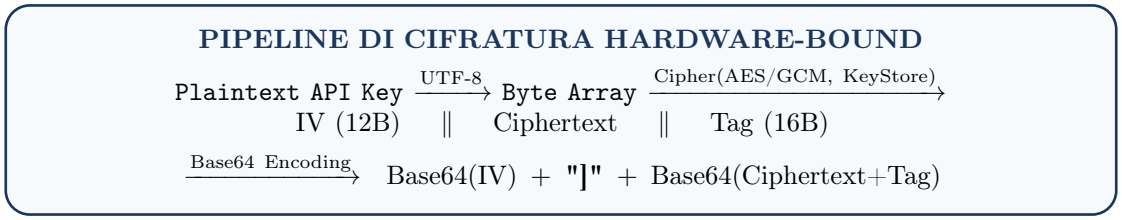


Figura 3: Struttura del payload cifrato serializzato.

5.2 Parametri Crittografici e Modalità Operativa

- **Provider Crittografico:** `AndroidKeyStore`;
- **Algoritmo Cifrante:** `AES` (Advanced Encryption Standard);
- **Dimensione Chiave:** 256 bit (32 byte);
- **Modalità di Blocco:** `GCM` (Galois/Counter Mode);
- **Padding:** `NoPadding`;
- **Autenticazione Tag:** 128 bit (16 byte).

Proprietà Hardware-Bound e Non-Esportabilità

Le master key con alias `simo_gemini_master_key` e `simo_openai_master_key` sono generate con il parametro:

`setRandomizedEncryptionRequired(true)`

La chiave non può mai lasciare il chip di sicurezza hardware. Se l'utente esporta un backup o se i dati dell'applicazione vengono clonati su un altro dispositivo, il token cifrato risulterà matematicamente indecifrabile (`AEADBadTagException`), garantendo l'assoluta impossibilità di esfiltrazione delle credenziali.

```
private fun getOrCreateSecretKey(): SecretKey {
    if (keyStore.containsAlias(KEY_ALIAS)) {
        val entry = keyStore.getEntry(KEY_ALIAS, null) as? KeyStore.
    SecretKeyEntry
        if (entry != null) return entry.secretKey
    }

    val keyGenerator = KeyGenerator.getInstance(
        KeyProperties.KEY_ALGORITHM_AES,
```

```
9           ANDROID_KEYSTORE_PROVIDER
10      )
11      val keyGenSpecBuilder = KeyGenParameterSpec.Builder(
12          KEY_ALIAS,
13          KeyProperties.PURPOSE_ENCRYPT or KeyProperties.PURPOSE_DECRYPT
14      )
15          .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
16          .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)
17          .setKeySize(256)
18          .setRandomizedEncryptionRequired(true)
19
20      keyGenerator.init(keyGenSpecBuilder.build())
21      return keyGenerator.generateKey()
22  }
```

Listing 9: Generazione della SecretKey hardware in CryptoUtils.kt

6 Integrazione Cloud AI: Architettura Dual-Engine (Gemini & ChatGPT)

6.1 Architettura Dual-Provider e Protocollo di Rete

Il sottosistema di intelligenza artificiale culinaria adotta un'astrazione modulare bivalente che supporta sia i modelli di **Google Gemini** (`GeminiRecipeService.kt`) sia i modelli di **OpenAI ChatGPT** (`OpenAiRecipeService.kt`):

- **Google Gemini Endpoint (REST v1beta):**

`POST https://generativelanguage.googleapis.com/v1beta/models/model:generateContent?key=apiKey`

- **OpenAI ChatGPT Endpoint (Chat Completions v1):**

`POST https://api.openai.com/v1/chat/completions (Header: Authorization Bearer apiKey)`

L'utente può alternare a caldo il motore preferito oppure delegare all'automa interno il fallback trasparente nel caso in cui uno dei servizi sia temporaneamente saturo o privo di credito residuo.

6.2 Selezione del Modello (Model Tiering)

Per massimizzare reattività e accuratezza culinaria, l'architettura supporta due modelli con scopi differenziati:

1. `gemini-flash-lite-latest` **(Modello Predefinito):**
   - Latenza media: $\sim 800 - 1400$ ms.
   - Utilizzo: Generazione rapida di piatti tradizionali, ricette standard, compendio ingredienti.
2. `gemini-flash-latest` **(Modello Avanzato per Preparazioni Complesse):**
   - Finestra di contesto e capacità di reasoning termodinamico e chimico avanzata.
   - Utilizzo: Grandi lievitati (panettoni, colombe), bilanciamento zuccheri e grassi nei gelati, gestione impasti ad alta idratazione ($\geq 80\%$).

6.3 Modalità Operative Non Distruttive dell'Assistente AI

L'interazione con l'Intelligenza Artificiale non si limita alla generazione integrale ex-novo di una ricetta, ma supporta interventi chirurgici non distruttivi volti a preservare il lavoro già svolto dallo chef:

1. **Genera Ricetta Completa:** Crea da zero la lista di ingredienti dosati, costi al chilo/litro, allergeni e procedimento numerato;
2. **Completa Dosi Mancanti (Non Distruttivo):** Analizza l'elenco di ingredienti già inseriti e compila **esclusivamente** le pesature non definite (quantità nulle o pari a zero), i prezzi mancanti, gli allergeni e i 6 parametri nutrizionali, senza alterare gli ingredienti o i dosaggi precedentemente stabiliti;
3. **Calcola Valori Nutrizionali (Preservazione Totale):** Esegue una stima analitica focalizzata esclusivamente sui 6 macronutrienti per 100g per ogni materia prima presente. Preserva intatti pesature, costi, allergeni e modificatore di resa termica;
4. **Genera Solo Procedimento Numerato:** Crea passaggi testuali sequenziali coerenti con le materie prime definite.

6.4 Prompt Engineering & Strutturazione JSON Forzata

Il prompt inviato al modello include vincoli contrattuali rigidi per assicurare una risposta conforme allo schema di parsing interno:

```
Tu sei un Maestro Pasticcere e Chef Professionista.
Genera una ricetta perfettamente bilanciata rispondendo ESCLUSIVAMENTE con un oggetto JSON valido.
Non aggiungere testo introduttivo, ne' spiegazioni, ne' blocchi di codice markdown.

Formato JSON richiesto:
{
  "name": "Nome ricetta",
  "category": "Dolci / Salati / Pane / Pizze / Primi / Secondi",
  "baseYieldQuantity": 10.0,
  "yieldUnit": "porzioni / cornetti / g / pizza",
  "description": "Breve descrizione",
  "instructions": "1. Primo passaggio...\n2. Secondo passaggio...",
  "components": [
    {
      "name": "Nome ingrediente",
      "quantity": 250.0,
      "unit": "g / ml / unita' / q.b.",
      "allergens": "GLUTINE,LATTE"
    }
  ]
}
```

Listing 10: Prompt di sistema con schema contrattuale JSON

6.5 Sanitizzazione Output & Motore di Fallback Euristico

La risposta del provider cloud attraversa un parser difensivo:

- **Detergente Markdown (`stripJsonMarkdown`):** Rimuove prefissi o suffissi del tipo ```` ```json ... ``` ````.

- **Offline Heuristic Generator:** Se la richiesta fallisce per assenza di connettività, quota API esaurita o formato malformato, si attiva automaticamente il generatore procedurale offline. Esso consulta un dizionario euristico interno per calcolare proporzioni classiche garantendo che l'utente non rimanga mai bloccato.

6.6 UI State Handling, Indicatori Lineari e Layout Scrollabile

Durante l'invocazione asincrona del modello generativo, lo stato è tracciato dal flag booleano `isAiLoading`. Per prevenire anomalie visive ed eccentricità di rendering riscontrabili nei circular spinner su schermi di ridotte dimensioni, l'interfaccia adotta un `LinearProgressIndicator` orizzontale a progressione vettoriale continua centrato all'interno di una card tematica. Inoltre, il contenitore modale integra il modificatore `Modifier.verticalScroll(rememberScrollState())`, garantendo la completa accessibilità dei controlli su qualsiasi diagonale di schermo senza incorrere in clipping o troncamento dei contenuti.

7 Cooking Mode, Concorrenza e Sottosistemi Hardware

7.1 Integrazione Window Manager: Flag "Keep Screen On"

Durante la sessione di preparazione, il display non deve spegnersi per timeout di inattività. L'implementazione evita permessi invasivi (`WAKE_LOCK`) ed esegue il binding sicuro al ciclo di vita del composable tramite `DisposableEffect`:

```
val currentView = LocalView.current
DisposableEffect(Unit) {
    currentView.keepScreenOn = true
    onDispose {
        currentView.keepScreenOn = false
    }
}
```

Listing 11: Binding dichiarativo del flag Keep-Screen-On in CookingModeScreen

7.2 Motore NLP Deterministico: `CookingTimerExtractor`

Il componente `CookingTimerExtractor.kt` scansiona le righe delle istruzioni applicando l'espressione regolare pre-compilata:

```
(?i)(?:per|circa)?\s*(\d+(?:[.,]\d+)?)...(ore|ora|h|minuti|min|m|secondi|s)\b
```

L'automa estrae il valore scalare, gestisce virgole e punti decimali, converte il periodo nell'intervallo $t \in [5, 86400]$ secondi e genera il pulsante interattivo associato al passaggio.

7.3 Macchina a Stati dei Timer Concorrenti e Architettura Persistente

Per garantire che il conto alla rovescia non subisca drift temporale o azzeramenti imprevisti quando l'applicazione viene posta in background (ad esempio durante l'ascolto o la visione di video su YouTube, o a fronte della liberazione della memoria di processo da parte dell'OS), la gestione temporale è disaccoppiata dal ciclo di refresh della UI e ancorata a timestamp assoluti di sistema.

I timer attivi sono modellati mediante la data class `ActiveKitchenTimer`:

```
data class ActiveKitchenTimer(
    val id: String = UUID.randomUUID().toString(),
    val label: String,
    val totalSeconds: Int,
    val remainingSeconds: Int = totalSeconds,
```

```
    val isRunning: Boolean = true,
    val isFinished: Boolean = false,
    val endTimeMillis: Long = System.currentTimeMillis() + (totalSeconds * 1000L),
    val pausedRemainingSeconds: Int? = null
) {
    fun currentRemainingSeconds(): Int {
        if (isFinished) return 0
        return if (!isRunning) {
            pausedRemainingSeconds ?: remainingSeconds
        } else {
            val remaining = ((endTimeMillis - System.currentTimeMillis()) / 1000).toInt()
            Math.max(0, remaining)
        }
    }
}
```

Listing 12: Modello dati immutabile del timer con epoch timestamp assoluto

La continuità operativa è presidiata dal singleton architetturale `CookingSessionManager`:

- **Persistenza dello Stato UI e Ingredienti:** Ogni variazione (ingredienti pesati, passaggi completati, timer aggiunti o rimossi) viene serializzata atomicamente in formato JSON all'interno di `SharedPreferences`. In caso di distruzione della schermata o riavvio forzato da parte del sistema operativo, la sessione viene ricaricata istantaneamente (`hasActiveSession()`), ripristinando fedelmente l'albero scalato e lo stato di avanzamento;
- **Foreground Service Dedicato (`CookingTimerService`):** Quando sono presenti timer in esecuzione, il sistema avvia un servizio di primo piano con notifica permanente (`startForegroundService`), che preserva il processo applicativo dallo scheduler a bassa priorità del kernel Linux e garantisce la tempestività dell'allarme sonoro anche con display bloccato.

7.4 Interfacciamento con Hardware HAL: Audio & Vibrazione

Al raggiungimento dello zero ($t = 0$), il sistema interroga l'HAL:

- **Audio:** Generazione deterministica tramite `ToneGenerator(AudioManager.STREAM_ALARM, 100)`, con emissione ciclica di `TONE_PROP_BEEP2`.
- **Vibrazione Multilivello:** Gestione congiunta di `VibratorManager` (per Android 12+, API ≥ 31) e fallback su `Vibrator` per le versioni legacy (API $24 - 30$).

7.5 Componente Modale Non Intrusivo: `QuickAddTimerDialog`

Il dialogo modale compatto per l'aggiunta di intervalli temporali discreti (**+30s; +1m; +5m; +10m**) opera direttamente sulla lista mutabile dei timer senza ricomporre l'albero di rendering delle istruzioni, garantendo transizioni fluide e tempo di frame ≤ 8 ms.

7.6 Gestione Layout Insets e Risoluzione Occlusioni di Sistema

Nei dispositivi moderni con navigazione gestuale a schermo intero (Android 10+, API ≥ 29), la barra inferiore dei timer (*Floating Multi-Timer Bar*) è potenzialmente soggetta a sovrapposizione con la barra di sistema (*gesture pill*). L'implementazione in `CookingModeScreen.kt` applica il padding dinamico derivato da `WindowInsets.navigationBars`, garantendo che tutti i componenti di controllo rimangano entro l'area utile (*safe area*) dello schermo. Inoltre, i pulsanti di avvio del timer applicano vincoli proporzionali per distribuire uniformemente lo spazio orizzontale su qualsiasi larghezza di display, prevenendo la compressione o l'interruzione di riga non voluta dei testi.

8 Rendering Grafico e Generazione PDF

8.1 Architettura del Modulo `RecipePdfExporter`

L'esportazione documentale non fa affidamento su librerie esterne pesanti (come iText o Apache PDFBox), bensì sfrutta la pipeline vettoriale nativa del sistema operativo: `android.graphics.pdf.PdfDocument` e la classe `Canvas`.

8.2 Geometria della Pagina e Mappatura Tipografica

Il layout rispetta le specifiche internazionali ISO 216 per il formato **A4** con risoluzione tipografica a 72 DPI:

- **Larghezza Pagina (W):** 595 pt (210 mm);
- **Altezza Pagina (H):** 842 pt (297 mm);
- **Margini Laterali (M):** 40 pt (14.1 mm);
- **Larghezza Area Utile (W_{content}):** $W - 2M = 515$ pt.

8.3 Architettura di Paginazione Deterministica (≥ 2 Pagine)

Per massimizzare la chiarezza in ambiente professionale e separare rigorosamente la distinta base dal protocollo esecutivo, l'esportatore implementa una suddivisione deterministica in almeno due pagine:

- **Pagina 1 (Distinta Materie Prime, Food Cost e Nutrizione UE):**
  - Intestazione grafica SIMO, data e timestamp di compilazione;
  - Metadati di ricetta: titolo, categoria, porzioni di riferimento e modificatore resa in cottura ($\Delta M\%$);
  - Tabella ingredienti scalati con quantità, unità, costo elementare e badge allergeni;
  - Riquadro contabile Food Cost (Costo totale impasto e Costo unitario a porzione);
  - Tabella grafica conforme al Reg. UE 1169/2011 con dicitura ufficiale di Stima, colonne per 100g e per porzione, energia (kJ/kcal), grassi, carboidrati, proteine, sale equivalente, sodio e acqua.
- **Pagina 2 (Procedimento di Preparazione e Istruzioni Operative):**
  - Riservata integralmente ai passaggi numerati di lavorazione;
  - Tipografia in nero pieno ad altissimo contrasto (`Color.BLACK`) con word-wrapping su canvas vettoriale;
  - Gestione del salto pagina automatico in caso di procedimenti straordinariamente articolati.

8.4 Algoritmo di Paginazione e Word-Wrapping Dinamico

Un problema tipico della stampa da mobile è la rottura di riga per passaggi procedurali lunghi. `RecipePdfExporter` implementa un motore di misura del testo:

```
private fun drawWrappedText(
    canvas: Canvas,
    text: String,
    x: Float,
    startY: Float,
    maxWidth: Float,
    paint: Paint,
    lineSpacing: Float
```

```
): Float {
    var currentY = startY
    val words = text.split(" ")
    var currentLine = StringBuilder()

    for (word in words) {
        val testLine = if (currentLine.isEmpty()) word else "$currentLine $word"
        val measuredWidth = paint.measureText(testLine)
        if (measuredWidth > maxWidth && currentLine.isNotEmpty()) {
            canvas.drawText(currentLine.toString(), x, currentY, paint)
            currentY += lineSpacing
            currentLine = StringBuilder(word)
        } else {
            currentLine = StringBuilder(testLine)
        }
    }
    if (currentLine.isNotEmpty()) {
        canvas.drawText(currentLine.toString(), x, currentY, paint)
        currentY += lineSpacing
    }
    return currentY
}
```

Listing 13: Word-wrapping dinamico su Canvas

8.5 Sandboxing e FileProvider Android

Il file compilato viene allocato nella directory di cache privata (`context.cacheDir/pdfs`) e condiviso verso le altre applicazioni mediante un Content URI firmato generato da `androidx.core.content.FileProvider`, conforme alle policy di sicurezza di Android 11+ (Scoped Storage).

9 Sottosistema di Backup, Serializzazione e Merge

9.1 Schema Canonico SIMO JSON v2

Il file di esportazione è strutturato per incapsulare atomicamente l'intero grafo del ricettario (Schema versione 2 con nutrizione e calo cottura):

```
{
  "version": 2,
  "appName": "SIMO",
  "timestamp": 1774028400000,
  "items": [
    {
      "id": "ing_farina_manitoba",
      "name": "Farina Manitoba",
      "type": "RAW_INGREDIENT",
      "defaultUnit": "g",
      "pricePerUnit": 1.60,
      "priceUnit": "kg",
      "allergens": "GLUTINE",
      "caloriesKcal": 364.0,
      "carbsG": 72.0,
      "fatsG": 1.2,
      "proteinsG": 14.5,
      "waterG": 12.0,
      "sodiumMg": 2.0
    }
  ],
```

```
  "recipes": [
    {
      "id": "recipe_cornetto_crema",
      "name": "Cornetto alla Crema",
      "baseYieldQuantity": 10.0,
      "yieldUnit": "cornetti",
      "category": "Dolci",
      "cookingWeightModifier": -10.0,
      "instructions": "...",
      "components": [
        {
          "childItemId": "ing_farina_manitoba",
          "baseQuantity": 210.0,
          "unit": "g"
        }
      ]
    }
  ]
}
```

Listing 14: Schema JSON canonico di backup Versione 2

9.2 Algoritmo di Risoluzione Conflitti a Due Fasi

Durante il ripristino (`importBackup`):

1. **Analisi Preventiva:** Il parser analizza il file senza scrivere su disco, confrontando `recipe.id` e `recipe.name` con il database locale per distinguere *Nuove Ricette* e *Conflitti*;
2. **Risoluzione e Commit Atomico:** L'utente sceglie **Sostituisci** (Replace) o **Mantieni** (Keep) per ogni conflitto. Il salvataggio avviene in una transazione SQLite isolata, garantendo l'integrità delle chiavi esterne.

10 Testing, Validazione e Qualità del Codice

La garanzia di qualità e l'integrità matematica di SIMO sono presidiate da una suite estesa di **59 test unitari automatizzati** eseguiti su host JVM con tempo di esecuzione totale $\leq 8\,\text{s}$.

Suite di Test	Casi di Test	Obiettivo di Validazione
`MathEngineTest`	7 test	Scaling proporzionale lineare, reverse scaling, calcolo food cost, tolleranze numeriche.
`NutritionCalculationTest`	6 test	Formule nutrizionali UE 1169/2011, conversione Sodio $\rightarrow$ Sale, densità a crudo.
`CookingWeightModifierTest`	5 test	Calo peso evaporativo, espansione per assorbimento, concentrazione nutrienti su massa cotta.
`CulinaryUnitEquivalenceTest`	4 test	Equivalenza $1:1$ tra massa e volume ($g \leftrightarrow ml$, $kg \leftrightarrow l$) nel calcolo dei prezzi.
`MathEngineSafetyTest`	5 test	Robustezza a floating-point estremi ($NaN; +\infty; -\infty$; divisioni per zero, grafi patologici).
`AllergenLogicTest`	4 test	Riconoscimento 14 allergeni UE, serializzazione stringhe CSV, ereditarietà sotto-ricette.
`AiPromptBuilderTest`	5 test	Preservazione unità intelligenti (torta, bicchieri, panini), vincoli non distruttivi.
`GeminiRecipeServiceTest`	4 test	Sanitizzazione regex JSON markdown, parsing difensivo, fallback offline euristico.
`OpenAiRecipeServiceTest`	4 test	Parsing Chat Completions HTTP, gestione errori quota e conformità token.
`ShoppingListLogicTest`	5 test	Aggregazione automatica per unità, somme ponderate, deduplicazione e persistenza.
`CookingTimerExtractorTest`	6 test	Pattern complessi ore/minuti/secondi, durate composte e tolleranze lessicali.
`NumberUtilsTest`	4 test	Formattazione valori numerici e conversione stringa-float sicura con locale US/IT.

Tabella 1: Riepilogo delle 12 suite di test automatizzate in SIMO (59 test complessivi).

Nei calcoli di scaling e food cost, i numeri in virgola mobile a doppia precisione (`Double`) sono soggetti a errori di cancellazione numerica o imprecisioni infinitesime di rappresentazione binaria (esempio: $0.1 + 0.2 \neq 0.3$). Il modulo di calcolo applica sistematicamente una funzione di tolleranza:

$$|a - b| < \epsilon, \qquad \text{con } \epsilon = 10^{-4}$$

Inoltre, qualsiasi fattore non finito (`!factor.isFinite() || factor <= 0.0`) viene automaticamente coercito al valore neutro unitario (1.0), impedendo la corruzione a cascata dello stato applicativo.

10.1 Pipeline di Distribuzione, Firma Crittografica e Keystore Personale

Per la distribuzione autonoma (*sideloading* e canali diretti) nel rispetto degli standard di sicurezza Android e dei controlli euristici di Google Play Protect, la compilazione del pacchetto APK di rilascio è vincolata a un Keystore crittografico dedicato (`simo-release.jks`):

- **Identità dello Sviluppatore Certificata:** Il certificato X.509 associato alla chiave RSA a 2048 bit è rilasciato con i metadati d'identità del creatore:

  `CN=Francesco Panattoni, OU=App Development, O=SIMO, C=IT`

  con impronta crittografica SHA-256 univoca memorizzata per la verifica di autenticità;

- **Firma Ibrida v1, v2 e v3 con `apksigner`:** Il build system Gradle (task `packageRelease`) applica sia la firma JAR classica (v1) sia la firma a blocchi dell'intero file APK (APK Signature Scheme v2 e v3). Ciò impedisce qualsiasi manomissione post-compilazione delle classi DEX o delle risorse binarie compresse;
- **Trasparenza e Mitigazione Play Protect:** L'adozione di un Keystore personale stabile assicura che gli aggiornamenti mantengano la continuità della catena di fiducia (*signing key rotation continuity*), eliminando falsi allarmi del motore Play Protect e garantendo all'utente finale un'installazione sicura e trasparente.

11 Conclusioni e Scalabilità Futura

11.1 Valutazione dei Risultati

L'architettura ingegnerizzata per SIMO ha pienamente soddisfatto gli obiettivi di progetto con la release 2.0:

- **Aderenza Architetturale:** Completa separazione tra strato di presentazione (Jetpack Compose), modello di dominio matematico (DAG Engine puro) e persistenza (Room SQLite);
- **Reattività Runtime:** Frame rate stabile a 60 FPS con zero jank percepibile durante il riparametramento continuo delle dosi e l'esecuzione contemporanea di timer multipli;
- **Conformità e Sicurezza:** Piena aderenza alla normativa UE 1169/2011 per gli allergeni e standard crittografico hardware-backed AES-256 GCM conforme ai requisiti OWASP Mobile Security.

11.2 Miglioramenti Futuri

Questa applicazione può essere ulteriormente migliorata aggiungendo ulteriori feature:

1. **Versione in Inglese:** Aggiunge un'impostazione per la lingua inglese in modo che l'app sia utilizzabile da più persone possibile;
2. **Integrazione Immagini:** Aggiunta di un supporto visivo per identificare rapidamente le ricette e implementazione dell'Intelligenza Artificiale per riconoscere ed estrarre preparazioni direttamente da una fotografia;
3. **Integrazione Comandi Vocali:** Utilizzando la voce sarà possibile inserire ricette, ma soprattutto interagire durante la Cooking Mode,
4. **Gestione Avanzata della Dispensa:** Filtro ricette realizzabili in tempo reale in base alle scorte disponibili, con modalità "svuota-frigo" assistita da AI per abbattere gli sprechi ed import automatico dagli articoli spuntati nella lista della spesa;
5. **Supporto Esteso a Nuovi Provider AI (Claude & DeepSeek):** Ulteriore arricchimento dell'architettura modulare bivalente per connettere modelli alternativi offline od on-premise;
6. **Integrazione Bilance Bluetooth Low Energy (BLE):** Interfacciamento diretto con le bilance da banco per l'acquisizione automatica del peso dell'ingrediente selezionato in Cooking Mode;
7. **Sincronizzazione Multi-Dispositivo Crittografata:** Estensione del backup verso archivi cloud end-to-end encrypted (E2EE) per la condivisione istantanea tra stazioni di lavoro e cucine multiple.